AI-ASSISTED DEVELOPMENT

How AI was used

Claude Code with SPEC-SS workflow

S P E C – S S

THE WORKFLOW

SPEC-SS, applied to every task

S

Specify

From PRD to datailed design docs

P

Plan

Turn design the into a plan

E

Execute

TDD

C

Check

Validate each user flow

S

Secure

Automated review

S

Ship

Open the PR and monitor CI

[S] SPECIFY

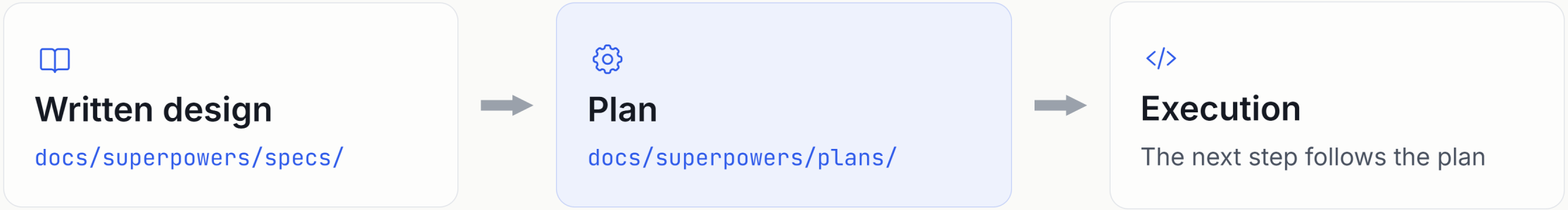
Before any code is written

01	**The issue** `/pmkit` Context, scope and acceptance criteria, from pmkit	`.issues/`
02	**Q&A session** `/grill-with-docs` `/superpowers:brainstorming` Sharpens the issue against the PRD and the domain glossary	`PRD · GLOSSARY.md`
03	**Terms and decisions recorded** When a new term or a hard-to-reverse decision comes up	`GLOSSARY.md · docs/adr/`
04	**Output: a written design doc** The session ends in a design document	`docs/superpowers/specs/`

[P] PLAN

A plan comes from the design

`/superpowers:writing-plans`



[E] EXECUTE

TDD

Implementation follows test-driven development. The skill depends on the size of the task:

SMALL TASKS

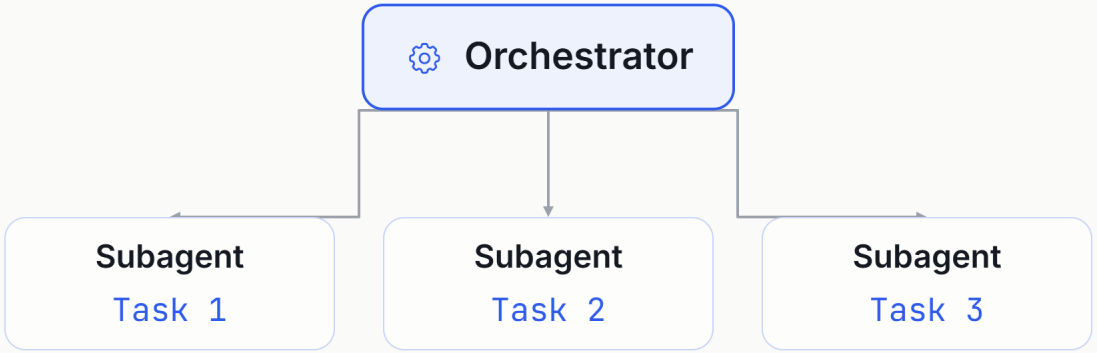
`/superpowers:executing-plans`

MEDIUM AND LARGER TASKS

`/superpowers:subagent-driven-development`

</> **Single agent**

Task 1 → Task 2 → Task 3



[C] CHECK

Every user flow is validated

/runbooks

User flow
Each flow is validated

→

Runbook
docs/runbooks/

→

E2E test script
Verifies the runbook

API **Vitest** Web app **Playwright**

[S] SECURE

Automated review before each pull request

`/utils:review-work`

The branch is checked against 12 criteria

01 Spec match	02 Scope and size	03 Readability and conventions	04 Tech debt
05 Unit tests	06 E2E tests	07 Security	08 Cyclomatic complexity
09 Error handling and edge cases	10 Performance	11 Backward compatibility	12 Documentation

[S] SHIP

Ship when every check passes

`/superpowers:finishing-a-development-branch`

ALL CHECKS PASS

- Tests
- Lint
- Typecheck
- Build

→

PR created

→

CI monitored